

Solutions for Homework Assignment #5

Answer to Question 1.

a. Let F be a CNF formula. Since F is a conjunction of clauses, it is clear that

$$F \text{ is a tautology if and only if every clause of } F \text{ is a tautology.} \quad (1)$$

Furthermore, since a clause C is a disjunction of literals,

$$\text{the clause } C \text{ is a tautology if and only if } C \text{ contains two opposite literals,} \quad (2)$$

where two literals are “opposite” if one of them is x and the other is $\neg x$ for the same variable x : If the clause contains two opposite literals, then every truth assignment satisfies one of these two literals, and therefore the entire disjunction; and if the clause does not contain two opposite literals, then it can be falsified by choosing a truth assignment that makes every literal false. From (1) and (2) we conclude that F is a tautology if and only if every clause contains two opposite literals. So the following algorithm is a decider for CNF-TAUT:

```
CNF-TAUT-DECIDER( $F$ )
1  for each clause  $C$  of  $F$  do
2     $taut := \text{false}$ 
3    for each pair of literals  $\ell$  and  $\ell'$  of  $C$  do
4      if one of  $\ell, \ell'$  is  $x$  and the other is  $\neg x$  for some variable  $x$  then  $taut := \text{true}$ ; break
5    if  $taut = \text{false}$  then reject
6  accept
```

The correctness of the algorithm follows from the preceding discussion. We now analyze its running time: Let n be the length of the given formula F , measured as the sum of the number of literals in all clauses. (A more detailed measure of the length of F would take into account the fact that each literal needs $O(\log_2 n)$ bits to be uniquely represented as a binary string. This, however, only increases the length by a factor of $\log_2 n$, so the two measures of length are polynomially related.) The formula has at most n clauses, and each clause has at most n literals and therefore at most n^2 pairs of literals. Checking whether two literals are opposite takes a constant amount of work ($O(\log_2 n)$ if we take into account the length of the representations of the literals, again only a factor of $\log_2 n$ more). So, the algorithm takes $O(n^3)$ time in a random access computer under the uniform cost criterion, and so it takes polynomial time on a deterministic Turing machine. Therefore $\text{CNF-TAUT} \in \mathbf{P}$, as wanted.

b. We first show that DNF-NONTAUT is in $\mathbf{NP}$ and then that it is $\mathbf{NP}$ -hard.

(i) $\text{DNF-NONTAUT} \in \mathbf{NP}$: We can prove this either using the definition of $\mathbf{NP}$ or the equivalent certificate-verifier characterization of $\mathbf{NP}$. For purposes of illustration we describe both approaches.

Argument using the definition of $\mathbf{NP}$: A NTM first uses non-determinism to write a truth assignment τ to the variables $x_1, x_2, \dots, x_n$ of the given formula F , “guessing” a truth value for each of these variables in each step; it then deterministically evaluates F under τ . If the formula is false under the truth assignment, it accepts; otherwise it rejects. Both the deterministic and nondeterministic stages of this NTM take polytime in $|F|$, so the NTM runs in polytime, implying that $\text{DNF-NONTAUT} \in \mathbf{NP}$.

Argument using the certificate-verifier characterization of $\mathbf{NP}$: The certificate is a truth assignment τ to the variables of the given formula F . The verifier evaluates F under τ to determine that F is false. A verifiable certificate exists if and only if F is not a tautology. The certificate is of polysize in $|F|$ and the verifier runs in polytime in $|F|$. Since DNF-NONTAUT has short and efficiently verifiable certificates, it is in $\mathbf{NP}$.

(ii) DNF-NONTAUT is NP-hard: To prove this it suffices to show that $\text{CNF-SAT} \leq_m^p \text{DNF-NONTAUT}$. To this end, let $F = (C_1 \wedge C_2 \wedge \dots \wedge C_m)$ be a CNF formula, where $C_j = (\ell_{j1} \vee \ell_{j2} \vee \dots \vee \ell_{jt_j})$ is the j -th clause of F , $j \in [1..m]$, and ℓ_{jk} is the k -th literal of clause C_j , $k \in [1..t_j]$. We have:

$$F \text{ is satisfiable} \Leftrightarrow \neg F \text{ is falsifiable} \Leftrightarrow \neg F \text{ is not a tautology.}$$

By applying DeMorgan's Laws, $\neg F$ is logically equivalent to the formula $\bar{F} = (T_1 \vee T_2 \vee \dots \vee T_m)$, where $T_j = (\overline{\ell_{j1}} \wedge \overline{\ell_{j2}} \wedge \dots \wedge \overline{\ell_{jt_j}})$ and $\overline{\ell_{jk}}$ is the opposite literal from ℓ_{jk} . Note that $\bar{F}$ is a DNF formula.

So the polytime reduction of CNF-SAT to DNF-NONTAUT is simply to map the given CNF formula F to the DNF formula $\bar{F}$, which can obviously be done in polynomial time.

Answer to Question 2.

a. We use the certificate-verifier characterization of **NP** to prove that $\text{HAMPATH} \in \text{NP}$. The certificate for a yes-instance $\langle G \rangle$, is a permutation of the nodes of G . Let n be the number of nodes of G , so this permutation can be described using $n \lceil \log_2 n \rceil$ bits. Given such a certificate, the verifier checks that (a) each node appears in the certificate exactly once, and (b) if u and v are successive nodes in the certificate, then (u, v) is an edge of G . This can be done (deterministically) in polynomial time. Since HAMPATH has short and efficiently verifiable certificates, it is in **NP**.

b. Given an instance $\langle G \rangle$ of HAMPATH, where $G = (V, E)$ is a directed graph, we show how to construct in polynomial time in the size of $\langle G \rangle$ a CNF propositional formula F_G such that

$$G \text{ has a Hamiltonian path} \Leftrightarrow F_G \text{ is satisfiable.} \quad (*)$$

Let $n = |V|$, $m = |E|$. The variables of formula F_G are (a) x_u^i , for each $u \in V$ and $i \in [1..n]$, and (b) e_{uv} for each $u, v \in V$. The intention is that x_u^i is true if and only if node u is the i -th node in a (supposed) Hamiltonian path of G ; and e_{uv} is true if and only if $(u, v) \in E$. (Note that we have a variable e_{uv} even if (u, v) is not an edge of G ; such variables will be forced to be false — see below.)

Intuitively, the formula F_G says that we can assign each node to precisely one of the n positions of a sequence of length n so that any two successive nodes in the sequence are connected by an edge. F_G is the conjunction of five subformulas, each expressing a relevant requirement.

(1) Each position $i \in [1..n]$ of the sequence is assigned a node:

$$F_G^1 = \bigwedge_{i \in [1..n]} \left(\bigvee_{u \in V} x_u^i \right).$$

(2) No position i of the sequence contains two different nodes u and v :

$$F_G^2 = \bigwedge_{i \in [1..n]} \left(\bigwedge_{\substack{u, v \in V \\ u \neq v}} (\neg x_u^i \vee \neg x_v^i) \right).$$

(3) Each node u appears in some position i of the sequence:

$$F_G^3 = \bigwedge_{u \in V} \left(\bigvee_{i \in [1..n]} x_u^i \right).$$

(4) If (u, v) is not an edge, then the variable e_{uv} cannot be true:

$$F_G^4 = \bigwedge_{\substack{u, v \in V \\ (u, v) \notin E}} \neg e_{uv}.$$

(5) If u is immediately followed by v in the sequence, then (u, v) must be edge:

$$F_G^5 = \bigwedge_{i \in [1..n-1]} \left(\bigwedge_{u,v \in V} (\neg x_u^i \vee \neg x_v^{i+1} \vee e_{uv}) \right).$$

The formula F_G is the conjunction of these five formulas: $F_G = F_G^1 \wedge F_G^2 \wedge F_G^3 \wedge F_G^4 \wedge F_G^5$. Since each subformula is in CNF so is their conjunction F_G .

Next we show that the length of the formula F_G is polynomial in the size of the given instance $\langle G \rangle$ of HAMPATH. Let $G = (V, E)$, $n = |V|$, $m = |E|$. The formula involves n^2 variables x_u^i and n^2 variables e_{uv} . We will measure the length of a CNF formula by the number of occurrences of literals (this is off by a factor of only $\log_2 n$ from the number of bits needed to represent the formula, so it is only polynomially shorter than that). We have:

$$\begin{aligned} |F_G^1| &= n \cdot n = O(n^2) \\ |F_G^2| &= n \cdot n \cdot (n-1) = O(n^3) \\ |F_G^3| &= n \cdot n = O(n^2) \\ |F_G^4| &= n^2 - m = O(n^2) \\ |F_G^5| &= (n-1) \cdot n^2 = O(n^3) \end{aligned}$$

Therefore $|F_G| = O(n^3)$ — i.e., the length of F_G is polynomial in the size of the given instance of HAMPATH. Constructing $\langle F_G \rangle$ from G is clearly polytime — to effect this construction, an algorithm simply writes down the formulas listed above, based on G . So, this can be done in polynomial time in a random access computer under the unit cost criterion, and therefore in polynomial time by a (deterministic) Turing machine.

Finally we prove (*).

[ONLY IF] Suppose that G has a Hamiltonian path $u_1, u_2, \dots, u_n$. Consider the truth assignment τ that makes x_u^i true if and only if u is the i -th node on this path, and makes e_{uv} true if and only if (u, v) is one of the edges of this path. More precisely,

$$\begin{aligned} \tau(x_u^i) &= \begin{cases} 1, & \text{if } u = u_i \\ 0, & \text{otherwise} \end{cases} & \text{for each } u \in V \text{ and } i \in [1..n] \\ \tau(e_{uv}) &= \begin{cases} 1, & \text{if } u = u_i \text{ and } v = u_{i+1} \text{ for some } i \in [1..n-1] \\ 0, & \text{otherwise} \end{cases} & \text{for each } u, v \in V. \end{aligned}$$

Since $u_1, u_2, \dots, u_n$ is a Hamiltonian path, τ satisfies each of $F_G^1, F_G^2, F_G^3, F_G^4$, and F_G^5 , and therefore it satisfies their conjunction, F_G .

[IF] Suppose F_G is satisfiable, and let τ be a truth assignment that satisfies F_G ; so it satisfies each of $F_G^1, F_G^2, F_G^3, F_G^4$, and F_G^5 . Think of x_u^i being true as meaning that u is the i -th node in a sequence of length n . Because τ satisfies F_G^1 and F_G^2 , for each $i \in [1..n]$ there is exactly one node u_i such that the variable x_u^i is true under τ , so there is exactly one node in each position i of the sequence. Because the truth assignment satisfies F_G^3 , each node of G is in some position of this sequence. Because it satisfies F_G^5 , if u and v are consecutive nodes in the sequence (i.e, for some $i \in [1..n-1]$, x_u^i and x_v^{i+1} are both true) τ must also make e_{uv} true, and because τ also satisfies F_G^4 , (u, v) must be an edge. So the nodes of G listed in the sequence $u_1, u_2, \dots, u_n$ form a Hamiltonian path. Therefore, if F_G is satisfiable, G has a Hamiltonian path.